المدرسة الوطنية للعلوم التطبيقية - بني ملال

ⵜⴰⵙⴷⴰⵡⵉⵜ ⵏ ⵜⵎⴰⵙⵙⴰⵏⵉⵏ ⵜⵉⵏⵎⵍⴰⵍ - ⴱⵏⵉ ⵎⵍⴰⵍ

Ecole Nationale des Sciences Appliquées - Béni Mellal

Rapport des travaux pratiques

Architecture Monolithique, Monolithe Modulaire, Architecture Distribuée et Architecture SOA

Filière : Intelligence Artificielle et Cybersécurité

Module :
Architecture Logicielle

Réalisé par :

Abderrahman MACHHOURI

Encadré par :
Pr. BE EL BAGHAZAOUI

Année universitaire : 2025-2026

Table des matières

Liste des figures **2**

Liste des tableaux **3**

1 TP4 : Architecture Orientée Services (SOA) **4**
1.1 Objectifs pédagogiques 4
1.2 Architecture du système 4
1.3 Lien vers Github 4
1.4 Étape 1 : Créer le serveur Eureka 5
1.4.1 Configuration 5
1.4.2 Classe principale 5
1.4.3 Vérification 5
1.5 Étape 2 : Créer le service Product 6
1.5.1 Configuration 6
1.5.2 Modèle de données 6
1.5.3 Repository 6
1.5.4 Service 7
1.5.5 Contrôleur REST 7
1.5.6 Activation Eureka Client 8
1.6 Étape 3 : Créer le service Order 9
1.6.1 Configuration 9
1.6.2 Modèle de données 9
1.6.3 Client Feign pour Product Service 9
1.6.4 DTO pour Product 10
1.6.5 Service 10
1.6.6 Contrôleur 12
1.6.7 Activation des clients Feign 12
1.7 Tests avec Postman 13
1.7.1 Test 1 : Créer un produit 13
1.7.2 Test 2 : Lister les produits 13
1.7.3 Test 3 : Créer une commande 14
1.7.4 Test 4 : Vérifier le stock mis à jour 15
1.7.5 Test 5 : Lister les commandes 15
1.8 Exercices pratiques 16
1.8.1 Exercice 1 : Mise à jour du stock 16
1.8.2 Exercice 2 : API Gateway 17
1.8.3 Exercice 3 : Gestion des erreurs 21
1.9 Conclusion Globale 24

Table des figures

1.1 POST - Créer un produit (réponse 201 Created) 13
1.2 GET - Lister tous les produits 14
1.3 POST - Créer une commande (status CONFIRMED) 14
1.4 GET - Stock mis à jour après commande (stock = 8) 15
1.5 GET - Lister toutes les commandes 16
1.6 Erreur 400 — Stock insuffisant 17
1.7 GET - Lister les produits via API Gateway (port 8080) 18
1.8 POST - Créer un produit via API Gateway (port 8080) 19
1.9 GET - Lister les commandes via API Gateway (port 8080) 20
1.10 POST - Créer une commande via API Gateway (port 8080) 20
1.11 Erreur 404 — Produit inexistant 23
1.12 Erreur 503 — Product Service indisponible (fallback activé) 23
1.13 Erreur 404 — Stock insuffisant 24

Liste des tableaux

1.1 Composants de l'architecture SOA 4
1.2 Scénarios d'erreur gérés dans Order Service 22

TP4 : Architecture Orientée Services (SOA)

1.1 Objectifs pédagogiques

- Comprendre les principes de l'architecture SOA
- Créer des microservices avec Spring Boot
- Implémenter la communication inter-services via REST
- Gérer une base de données avec Spring Data JPA
- Utiliser un service de découverte (Eureka)
- Implémenter une API Gateway avec Spring Cloud Gateway
- Gérer les erreurs distribuées avec Resilience4j Circuit Breaker

1.2 Architecture du système

Notre application e-commerce est composée de quatre microservices indépendants, chacun avec sa propre base de données H2 en mémoire :

Table 1.1 – Composants de l'architecture SOA

Service	**Port**	**Description**
eureka-server	8761	Service de découverte — registre de tous les microservices
product-service	8081	Gestion du catalogue produits (CRUD + gestion de stock)
order-service	8082	Gestion des commandes avec communication Feign vers Product Service
api-gateway	8080	Point d'entrée unique — route les requêtes via Spring Cloud Gateway

Chaque service s'enregistre automatiquement dans Eureka au démarrage. L'API Gateway utilise le load balancer (`lb://`) pour résoudre les services par leur nom logique.

1.3 Lien vers Github

Le code source complet de ce TP est disponible sur GitHub :
https://github.com/abderrahmanmachhouri/TPs_Genie_Logicielle/tree/main/TP4_soa_ecommerce

1.4 Étape 1 : Créer le serveur Eureka

Le serveur Eureka est un service de découverte qui permet aux microservices de s'enregistrer et de se trouver mutuellement.

1.4.1 Configuration

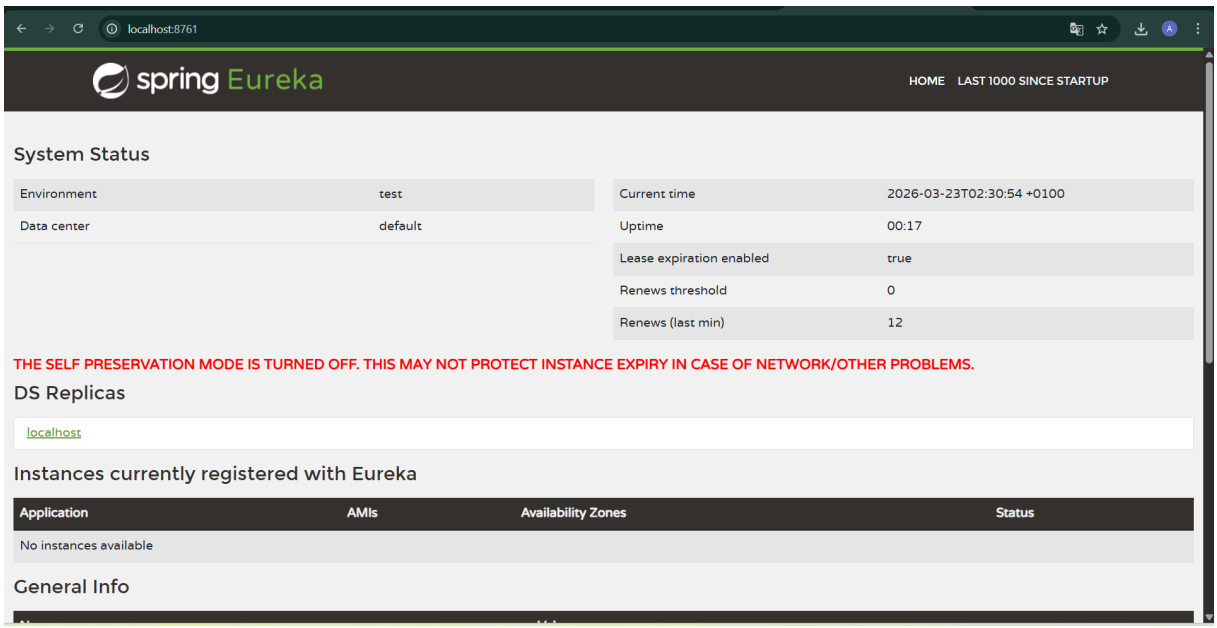
```
server.port=8761
spring.application.name=eureka-server
eureka.client.register-with-eureka=false
eureka.client.fetch-registry=false
eureka.client.service-url.defaultZone=http://localhost:8761/eureka/
eureka.server.enable-self-preservation=false
```

1.4.2 Classe principale

```
@SpringBootApplication
@EnableEurekaServer
public class EurekaServerApplication {
    public static void main(String[] args) {
        SpringApplication.run(EurekaServerApplication.class, args);
    }
}
```

1.4.3 Vérification

Après démarrage, ouvrir `http://localhost:8761` pour accéder au tableau de bord Eureka.



1.5 Étape 2 : Créer le service Product

1.5.1 Configuration

```
server.port=8081
spring.application.name=product-service
eureka.client.service-url.defaultZone=http://localhost:8761/eureka/
spring.datasource.url=jdbc:h2:mem:productdb
spring.datasource.driver-class-name=org.h2.Driver
spring.jpa.hibernate.ddl-auto=update
spring.h2.console.enabled=true
```

1.5.2 Modèle de données

```
@Entity
@Data
@NoArgsConstructor
@AllArgsConstructor
@Table(name = "products")
public class Product {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @NotBlank(message = "Le nom du produit est obligatoire")
    private String name;

    private String description;

    @NotNull
    @DecimalMin(value = "0.0", inclusive = false)
    private Double price;

    @NotNull
    @Min(value = 0)
    private Integer stock;
}
```

1.5.3 Repository

```
@Repository
public interface ProductRepository extends JpaRepository<Product,
   Long> {
    List<Product> findByNameContainingIgnoreCase(String name);
    List<Product> findByStockGreaterThan(Integer stock);
}
```

1.5.4 Service

```java
@Service
@RequiredArgsConstructor
@Slf4j
public class ProductService {

    private final ProductRepository productRepository;

    public List<Product> getAllProducts() {
        return productRepository.findAll();
    }

    public Product getProductById(Long id) {
        return productRepository.findById(id)
                .orElseThrow(() -> new ProductNotFoundException(id)
                    );
    }

    public Product createProduct(Product product) {
        return productRepository.save(product);
    }

    public Product updateProduct(Long id, Product details) {
        Product product = getProductById(id);
        product.setName(details.getName());
        product.setDescription(details.getDescription());
        product.setPrice(details.getPrice());
        product.setStock(details.getStock());
        return productRepository.save(product);
    }

    public void deleteProduct(Long id) {
        productRepository.delete(getProductById(id));
    }
}
```

1.5.5 Contrôleur REST

Les endpoints exposés par le Product Service :

- `GET /api/products` — lister tous les produits
- `GET /api/products/{id}` — récupérer un produit par ID
- `POST /api/products` — créer un produit
- `PUT /api/products/{id}` — mettre à jour un produit
- `DELETE /api/products/{id}` — supprimer un produit
- `PUT /api/products/{id}/stock` — mettre à jour le stock (Exercice 1)

```java
@RestController
@RequestMapping("/api/products")
@RequiredArgsConstructor
public class ProductController {
```

```java

    private final ProductService productService;

    @GetMapping
    public ResponseEntity<List<Product>> getAllProducts() {
        return ResponseEntity.ok(productService.getAllProducts());
    }

    @GetMapping("/{id}")
    public ResponseEntity<Product> getProductById(@PathVariable Long id) {
        return ResponseEntity.ok(productService.getProductById(id));
    }

    @PostMapping
    public ResponseEntity<Product> createProduct(
            @Valid @RequestBody Product product) {
        return ResponseEntity
                .status(HttpStatus.CREATED)
                .body(productService.createProduct(product));
    }

    @PutMapping("/{id}")
    public ResponseEntity<Product> updateProduct(
            @PathVariable Long id,
            @Valid @RequestBody Product product) {
        return ResponseEntity.ok(productService.updateProduct(id, product));
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> deleteProduct(@PathVariable Long id) {
        productService.deleteProduct(id);
        return ResponseEntity.noContent().build();
    }

    @PutMapping("/{id}/stock")
    public ResponseEntity<Product> updateStock(
            @PathVariable Long id,
            @RequestBody Map<String, Integer> body) {
        return ResponseEntity.ok(
                productService.updateStock(id, body.get("quantity"));
    }
}
```

1.5.6 Activation Eureka Client

```java
@SpringBootApplication
@EnableDiscoveryClient
public class ProductServiceApplication {
    public static void main(String[] args) {
        SpringApplication.run(ProductServiceApplication.class, args
            );
    }
}
```

1.6 Étape 3 : Créer le service Order

1.6.1 Configuration

```properties
server.port=8082
spring.application.name=order-service
eureka.client.service-url.defaultZone=http://localhost:8761/eureka/
spring.datasource.url=jdbc:h2:mem:orderdb
spring.datasource.driver-class-name=org.h2.Driver
spring.jpa.hibernate.ddl-auto=update
spring.h2.console.enabled=true
spring.cloud.openfeign.circuitbreaker.enabled=true
```

1.6.2 Modèle de données

```java
@Entity
@Data
@NoArgsConstructor
@AllArgsConstructor
@Table(name = "orders")
public class Order {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private Long productId;
    private String productName;
    private Integer quantity;
    private Double totalPrice;
    private LocalDateTime orderDate;
    private String status; // PENDING, CONFIRMED, CANCELLED
}
```

1.6.3 Client Feign pour Product Service

```java
@FeignClient(name = "product-service",
             fallback = ProductClientFallback.class)
public interface ProductClient {

    @GetMapping("/api/products/{id}")
    ProductDTO getProductById(@PathVariable Long id);

    @PutMapping("/api/products/{id}/stock")
    ProductDTO updateStock(@PathVariable Long id,
                           @RequestBody Map<String, Integer> body);
}
```

1.6.4 DTO pour Product

```java
@Data
@NoArgsConstructor
@AllArgsConstructor
public class ProductDTO {
    private Long id;
    private String name;
    private String description;
    private Double price;
    private Integer stock;
}
```

1.6.5 Service

```java
@Service
@RequiredArgsConstructor
@Slf4j
public class OrderService {

    private final OrderRepository orderRepository;
    private final ProductClient productClient;

    public List<Order> getAllOrders() {
        return orderRepository.findAll();
    }

    @Transactional
    public Order createOrder(Long productId, Integer quantity) {
        // 1. Recuperer le produit (Exercice 3 : gestion des
           erreurs)
        ProductDTO product = getProductSafely(productId);

        // 2. Verifier le stock (Exercice 1)
        if (product.getStock() < quantity) {
            throw new InsufficientStockException(productId,
               quantity);
```

```java
        }

        // 3. Creer la commande
        Order order = new Order();
        order.setProductId(productId);
        order.setProductName(product.getName());
        order.setQuantity(quantity);
        order.setTotalPrice(product.getPrice() * quantity);
        order.setOrderDate(LocalDateTime.now());
        order.setStatus("PENDING");
        Order saved = orderRepository.save(order);

        // 4. Mettre a jour le stock via Feign (Exercice 1)
        updateProductStockSafely(productId, quantity, saved.getId()
            );
        saved.setStatus("CONFIRMED");
        return orderRepository.save(saved);
    }

    private ProductDTO getProductSafely(Long productId) {
        try {
            ProductDTO product = productClient.getProductById(
                productId);
            if (product == null) {
                throw new ProductServiceException(
                    "Le service produit est indisponible.");
            }
            return product;
        } catch (FeignException.NotFound e) {
            throw e;
        } catch (FeignException e) {
            throw new ProductServiceException(
                "Impossible de contacter le service produit.", e);
        }
    }

    private void updateProductStockSafely(Long productId,
                                          Integer quantity,
                                          Long orderId) {
        try {
            productClient.updateStock(productId,
                    Map.of("quantity", quantity));
        } catch (Exception e) {
            orderRepository.findById(orderId).ifPresent(o -> {
                o.setStatus("CANCELLED");
                orderRepository.save(o);
            });
            throw new ProductServiceException(
                "Erreur de mise a jour du stock. Commande annulee."
                    , e);
        }
```

```
    }
}
```

1.6.6 Contrôleur

```
@RestController
@RequestMapping("/api/orders")
@RequiredArgsConstructor
public class OrderController {

    private final OrderService orderService;

    @GetMapping
    public ResponseEntity<List<Order>> getAllOrders() {
        return ResponseEntity.ok(orderService.getAllOrders());
    }

    @GetMapping("/{id}")
    public ResponseEntity<Order> getOrderById(@PathVariable Long id
        ) {
        return ResponseEntity.ok(orderService.getOrderById(id));
    }

    @PostMapping
    public ResponseEntity<Order> createOrder(
            @RequestParam Long productId,
            @RequestParam Integer quantity) {
        return ResponseEntity
                .status(HttpStatus.CREATED)
                .body(orderService.createOrder(productId, quantity)
                    );
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> cancelOrder(@PathVariable Long id)
        {
        orderService.cancelOrder(id);
        return ResponseEntity.noContent().build();
    }
}
```

1.6.7 Activation des clients Feign

```
@SpringBootApplication
@EnableDiscoveryClient
@EnableFeignClients
public class OrderServiceApplication {
    public static void main(String[] args) {
        SpringApplication.run(OrderServiceApplication.class, args);
```

```
    }
}
```

1.7 Tests avec Postman

1.7.1 Test 1 : Créer un produit

```
POST http://localhost:8081/api/products
Content-Type: application/json

{
  "name": "Laptop",
  "description": "Ordinateur portable haute performance",
  "price": 1200.00,
  "stock": 10
}
```

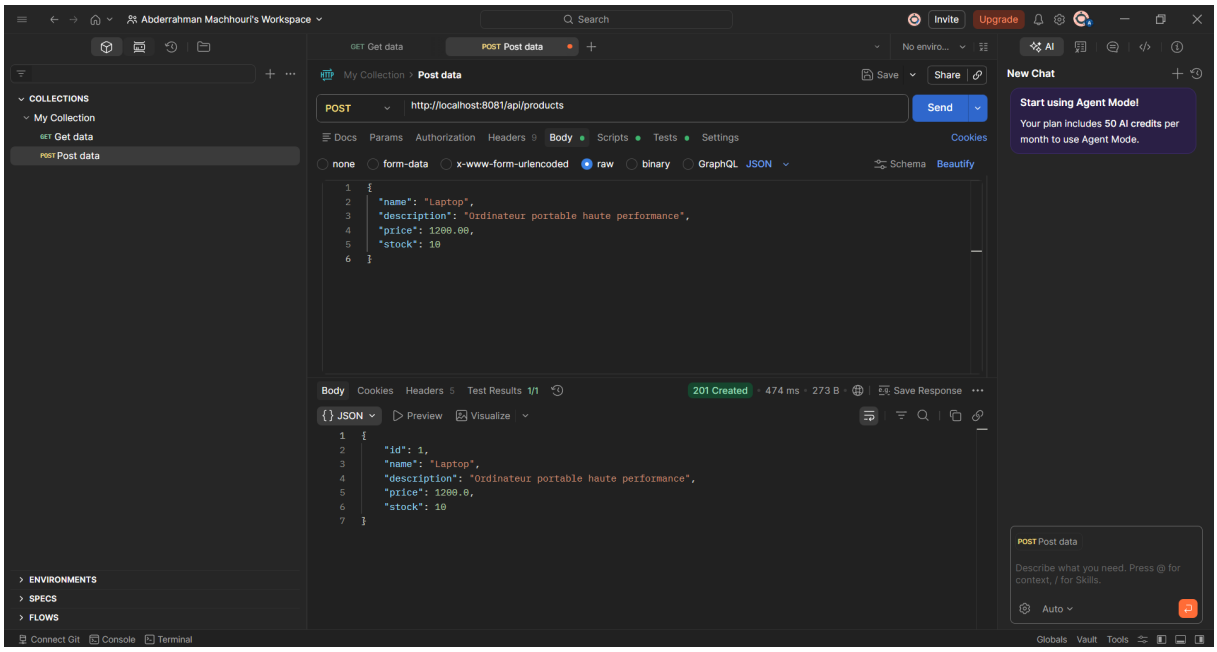


FIGURE 1.1 – POST - Créer un produit (réponse 201 Created)

1.7.2 Test 2 : Lister les produits

```
GET http://localhost:8081/api/products
```

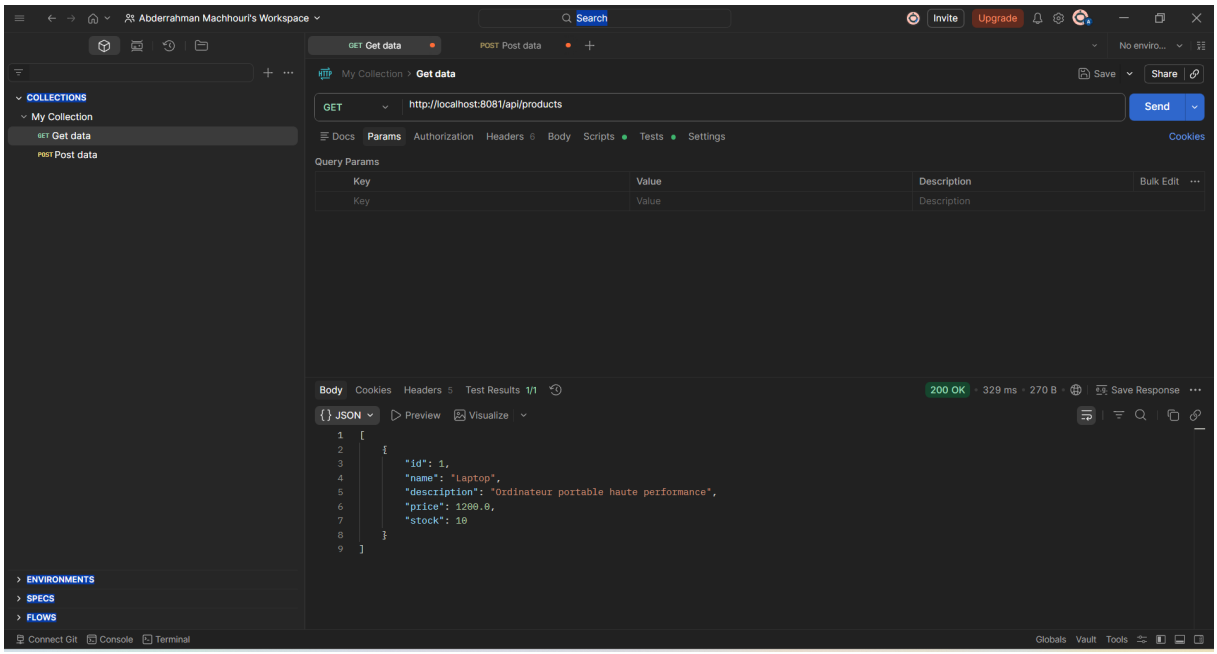


FIGURE 1.2 – GET - Lister tous les produits

1.7.3 Test 3 : Créer une commande

```
POST http://localhost:8082/api/orders?productId=1&quantity=2
```

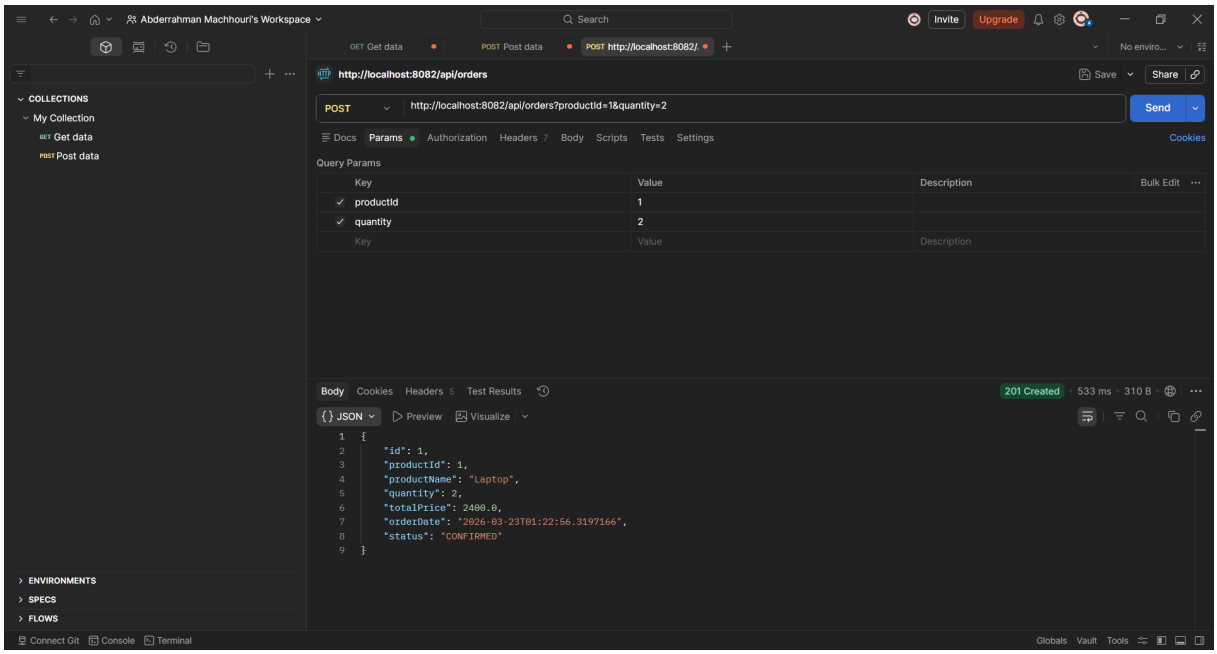


FIGURE 1.3 – POST - Créer une commande (status CONFIRMED)

1.7.4 Test 4 : Vérifier le stock mis à jour

Après la commande de 2 unités, le stock doit passer de 10 à 8 :

```
GET http://localhost:8081/api/products/1
```

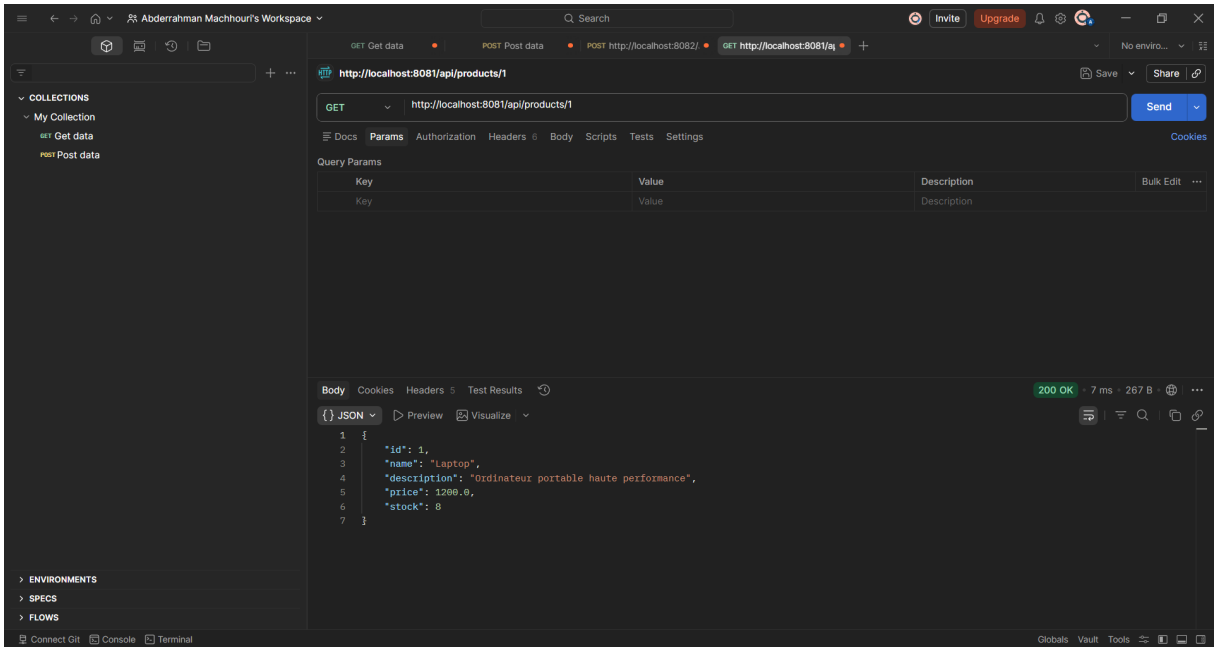


FIGURE 1.4 – GET - Stock mis à jour après commande (stock = 8)

1.7.5 Test 5 : Lister les commandes

```
GET http://localhost:8082/api/orders
```

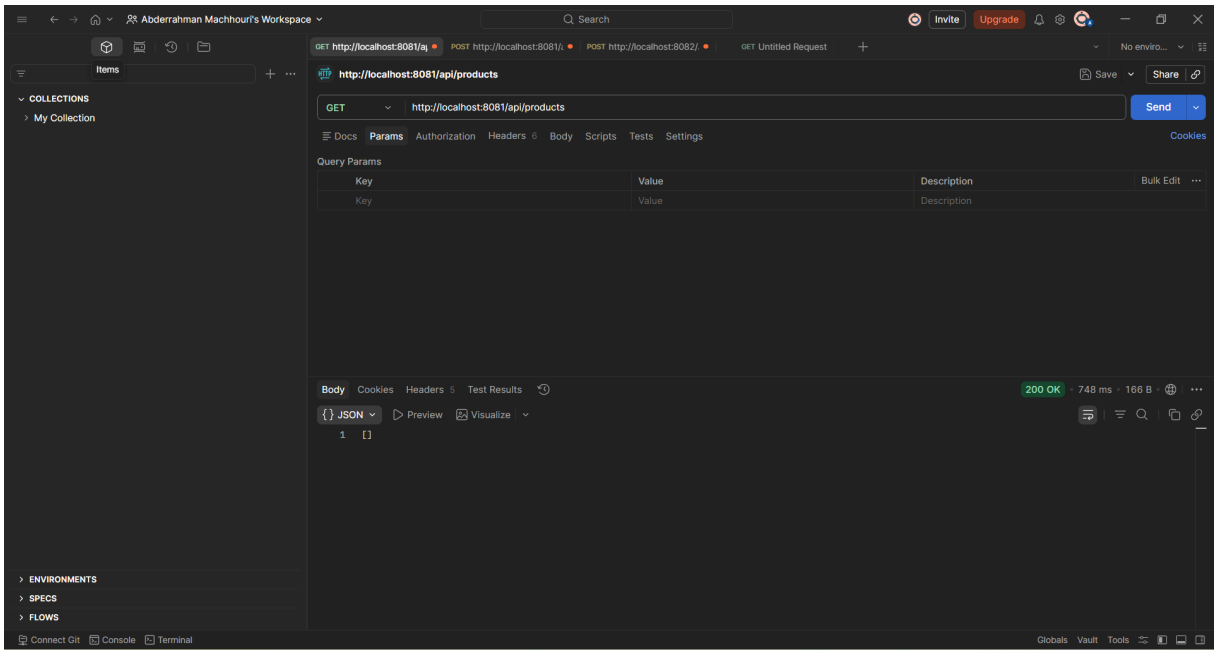


FIGURE 1.5 – GET - Lister toutes les commandes

1.8 Exercices pratiques

1.8.1 Exercice 1 : Mise à jour du stock

Cet exercice ajoute la gestion automatique du stock lors de la création d'une commande.

Endpoint ajouté dans Product Service

```
@Transactional
public Product updateStock(Long id, Integer quantityToReduce) {
    Product product = getProductById(id);
    if (product.getStock() < quantityToReduce) {
        throw new InsufficientStockException(
                id, quantityToReduce, product.getStock());
    }
    product.setStock(product.getStock() - quantityToReduce);
    return productRepository.save(product);
}
```

Test : stock insuffisant

On tente de commander 9999 unités alors que le stock est de 8 :

```
POST http://localhost:8082/api/orders?productId=1&quantity=9999
```

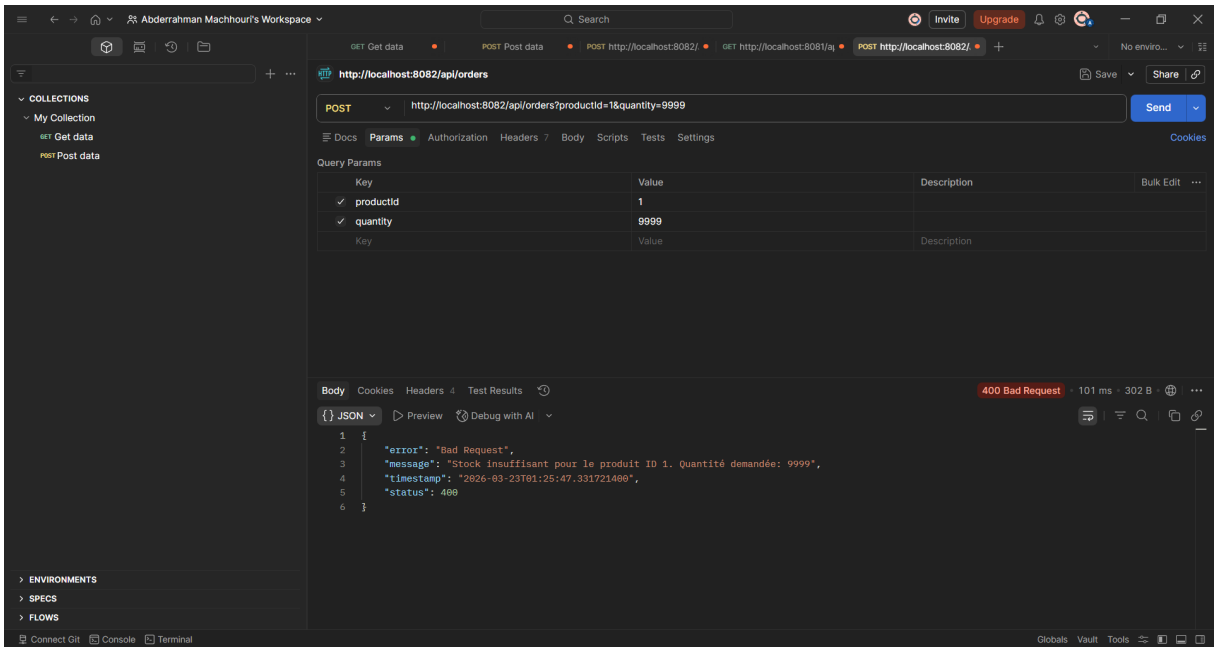


FIGURE 1.6 – Erreur 400 — Stock insuffisant

1.8.2 Exercice 2 : API Gateway

Configuration (application.yml)

```yaml
server:
  port: 8080

spring:
  application:
    name: api-gateway
  cloud:
    gateway:
      routes:
        - id: product-service
          uri: lb://product-service
          predicates:
            - Path=/products/**
          filters:
            - RewritePath=/products/(?<segment>.*),
                          /api/products/${segment}

        - id: order-service
          uri: lb://order-service
          predicates:
            - Path=/orders/**
          filters:
            - RewritePath=/orders/(?<segment>.*),
                          /api/orders/${segment}

```

```yaml
eureka:
  client:
    service-url:
      defaultZone: http://localhost:8761/eureka/
```

Classe principale

```java
@SpringBootApplication
@EnableDiscoveryClient
public class ApiGatewayApplication {
    public static void main(String[] args) {
        SpringApplication.run(ApiGatewayApplication.class, args);
    }
}
```

Test 1 : Lister les produits via Gateway

```
GET http://localhost:8080/products
```

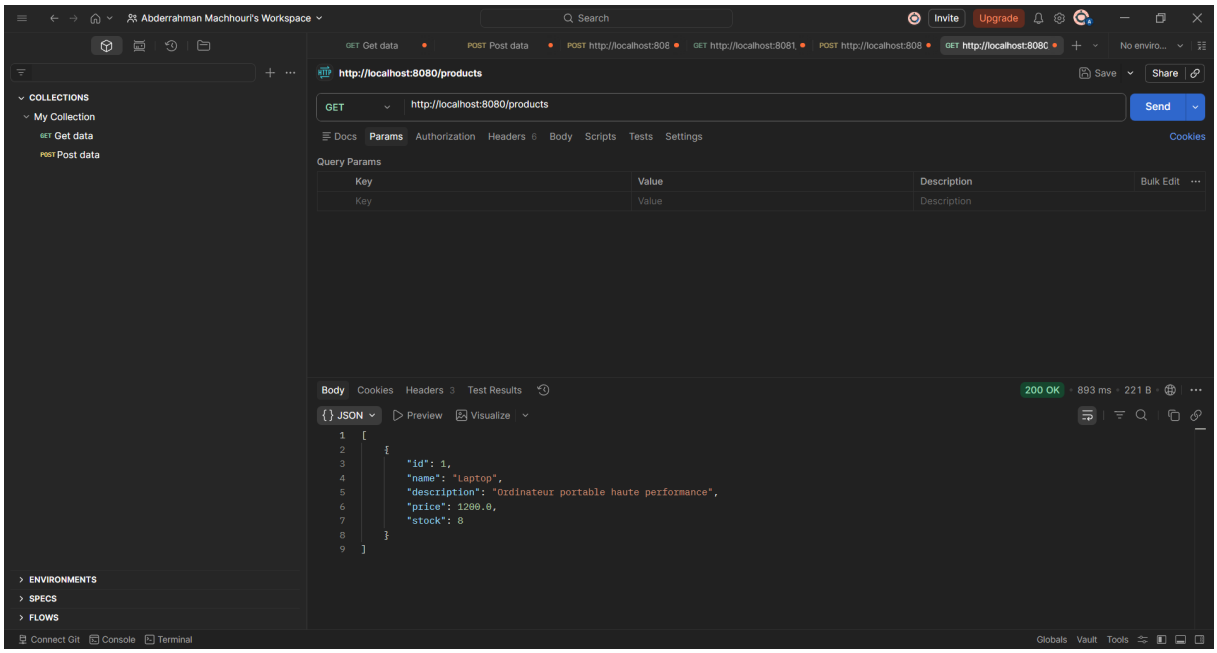


FIGURE 1.7 – GET - Lister les produits via API Gateway (port 8080)

Test 2 : Créer un produit via Gateway

```
POST http://localhost:8080/products
Content-Type: application/json

{
  "name": "Souris",
```

```
  "description": "Souris sans fil",
  "price": 35.00,
  "stock": 50
}
```

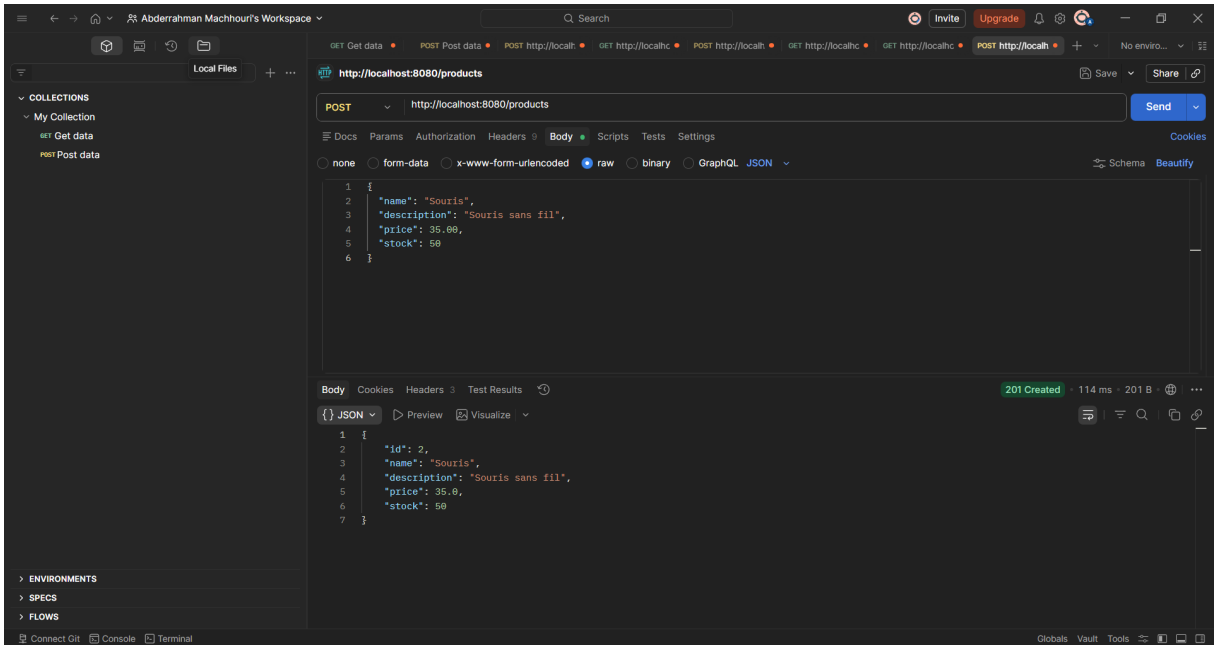


FIGURE 1.8 – POST - Créer un produit via API Gateway (port 8080)

Test 3 : Lister les commandes via Gateway

```
GET http://localhost:8080/orders
```

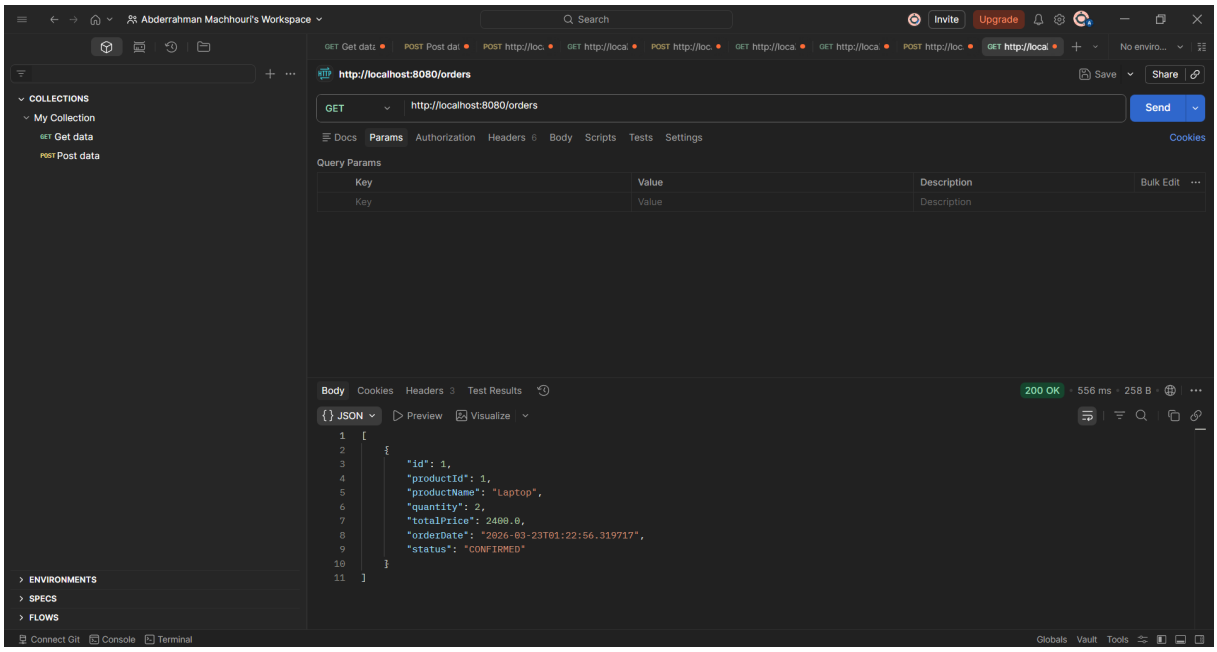


FIGURE 1.9 – GET - Lister les commandes via API Gateway (port 8080)

Test 4 : Créer une commande via Gateway

```
POST http://localhost:8080/orders?productId=1&quantity=1
```

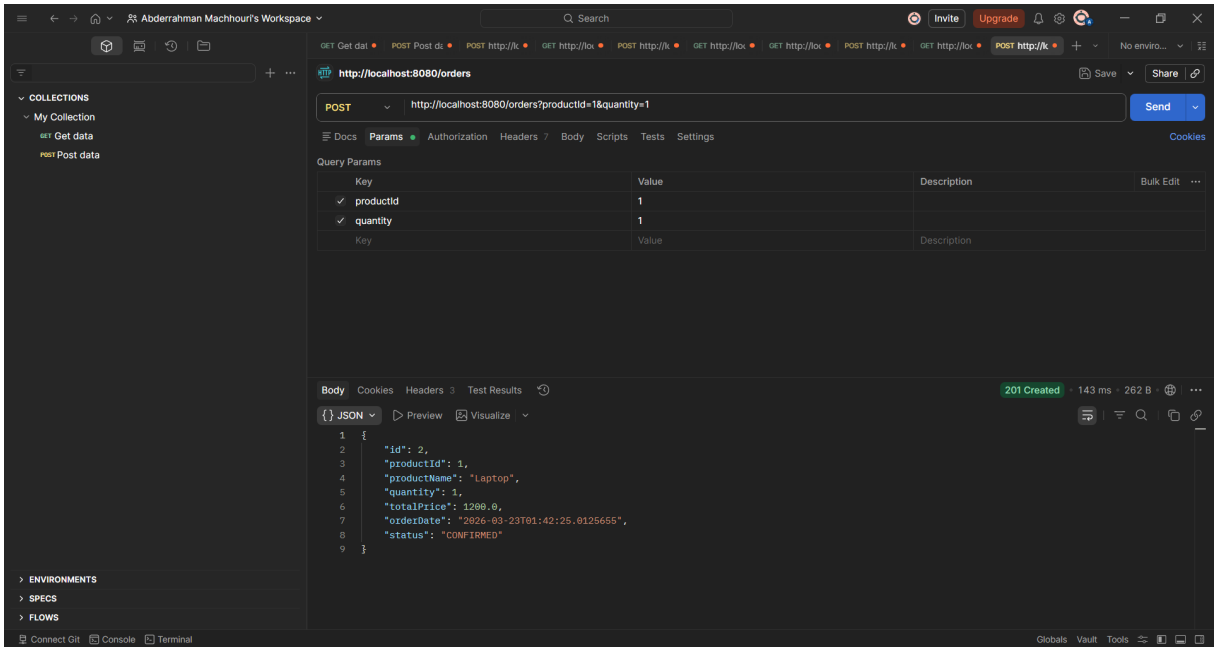


FIGURE 1.10 – POST - Créer une commande via API Gateway (port 8080)

1.8.3 Exercice 3 : Gestion des erreurs

Fallback Feign

Lorsque Product Service est indisponible, le fallback retourne `null`, ce qui déclenche une `ProductServiceException` avec le code HTTP 503 :

```
@Component
@Slf4j
public class ProductClientFallback implements ProductClient {

    @Override
    public ProductDTO getProductById(Long id) {
        log.error("Product Service indisponible - fallback ID {}",
           id);
        return null;
    }

    @Override
    public ProductDTO updateStock(Long id,
                                  Map<String, Integer> body) {
        log.error("Product Service indisponible - fallback stock {}
           ", id);
        return null;
    }
}
```

GlobalExceptionHandler

Un `@RestControllerAdvice` centralise la gestion des erreurs et retourne des réponses JSON structurées :

```
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(OrderNotFoundException.class)
    public ResponseEntity<Map<String, Object>> handleNotFound(
            OrderNotFoundException ex) {
        return buildError(HttpStatus.NOT_FOUND, ex.getMessage());
    }

    @ExceptionHandler(InsufficientStockException.class)
    public ResponseEntity<Map<String, Object>> handleStock(
            InsufficientStockException ex) {
        return buildError(HttpStatus.BAD_REQUEST, ex.getMessage());
    }

    @ExceptionHandler(ProductServiceException.class)
    public ResponseEntity<Map<String, Object>> handleUnavailable(
            ProductServiceException ex) {
        return buildError(HttpStatus.SERVICE_UNAVAILABLE,
                "Le service produit est temporairement indisponible
                   .");
```

```
    }

    @ExceptionHandler(FeignException.NotFound.class)
    public ResponseEntity<Map<String, Object>> handleFeignNotFound(
            FeignException.NotFound ex) {
        return buildError(HttpStatus.NOT_FOUND,
                "Le produit demande est introuvable.");
    }

    private ResponseEntity<Map<String, Object>> buildError(
            HttpStatus status, String message) {
        Map<String, Object> error = new HashMap<>();
        error.put("timestamp", LocalDateTime.now().toString());
        error.put("status", status.value());
        error.put("error", status.getReasonPhrase());
        error.put("message", message);
        return ResponseEntity.status(status).body(error);
    }
}
```

Tableau des scénarios d'erreur

Table 1.2 – Scénarios d'erreur gérés dans Order Service

Scénario	Code HTTP	Message retourné
Produit inexistant	404	Le produit demandé est introuvable
Stock insuffisant	400	Stock insuffisant pour le produit ID X
Product Service indisponible	503	Le service produit est temporairement indisponible
Champ de validation manquant	400	Erreur de validation + détails par champ

Test : produit inexistant

```
POST http://localhost:8082/api/orders?productId=9999&quantity=1
```

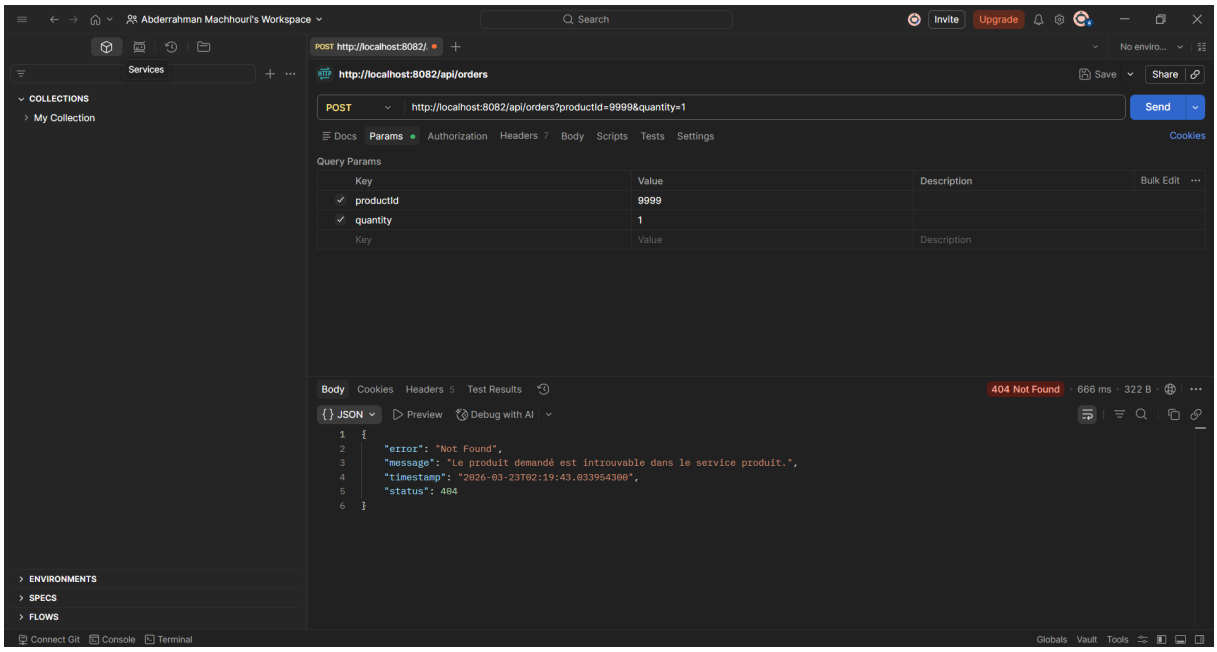


FIGURE 1.11 – Erreur 404 — Produit inexistant

Test : service Product indisponible

On arrête Product Service, puis on tente de créer une commande :

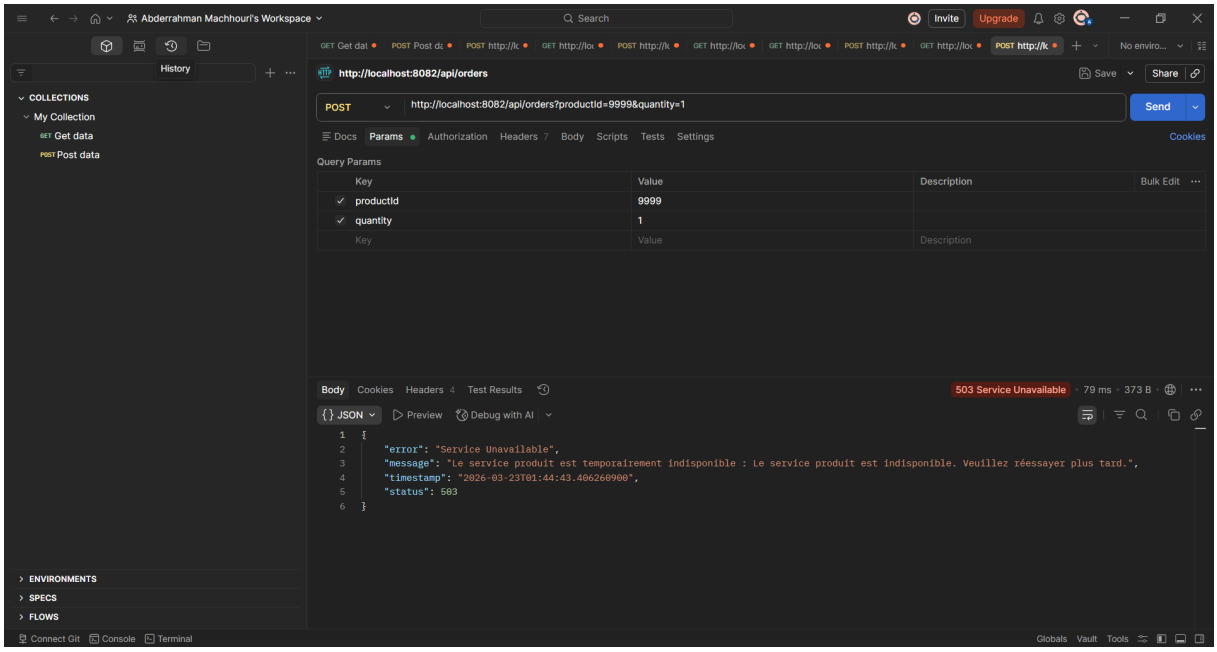


FIGURE 1.12 – Erreur 503 — Product Service indisponible (fallback activé)

Test : stock insuffisant

```
POST http://localhost:8082/api/orders?productId=1&quantity=9999
```

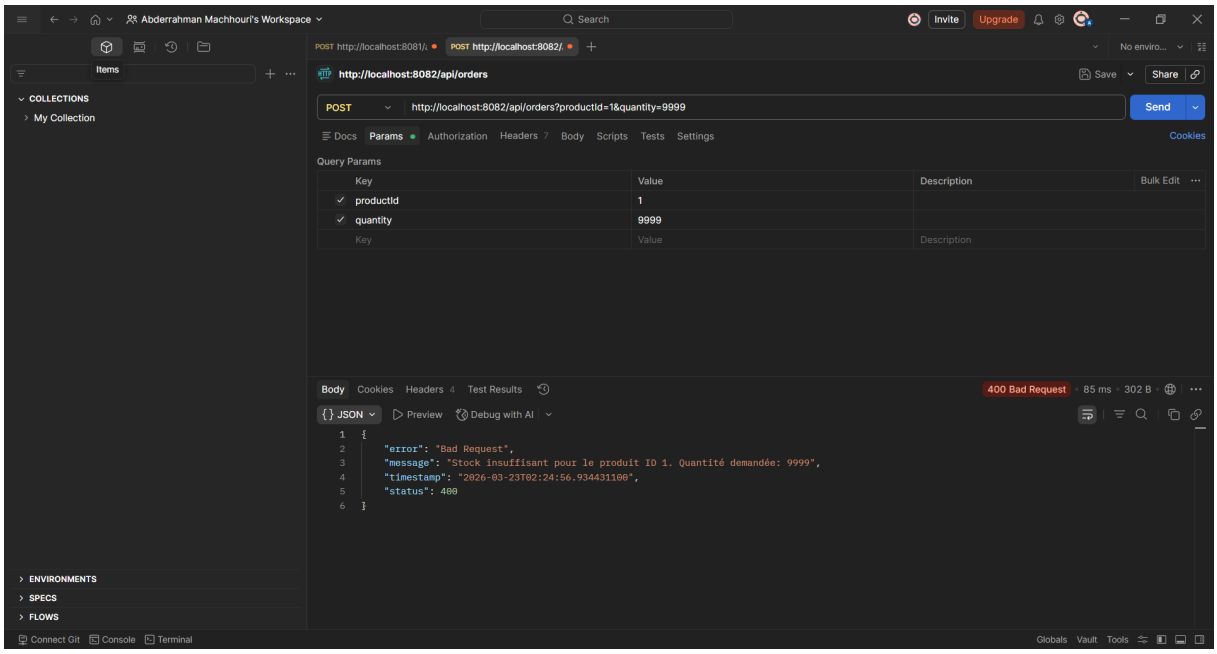


FIGURE 1.13 – Erreur 404 — Stock insuffisant

1.9 Conclusion Globale

Ce TP4 a permis de mettre en œuvre une architecture orientée services (SOA) complète à l'aide de Spring Boot et Spring Cloud. Quatre microservices indépendants ont été développés : Eureka Server pour la découverte de services, Product Service et Order Service pour la logique métier, et API Gateway comme point d'entrée unique.

Les exercices pratiques ont enrichi progressivement le projet. L'Exercice 1 a introduit la gestion automatique du stock inter-services via un endpoint dédié et une communication Feign transactionnelle. L'Exercice 2 a centralisé les accès avec Spring Cloud Gateway en exploitant la résolution des services par nom via Eureka. Enfin, l'Exercice 3 a renforcé la résilience du système en gérant les cas d'erreurs distribués (service indisponible, produit inexistant, stock insuffisant) grâce à Resilience4j Circuit Breaker et un `GlobalExceptionHandler`.

Par rapport aux TP précédents, cette approche SOA offre une meilleure scalabilité (chaque service peut être déployé indépendamment), une isolation des pannes grâce au mécanisme de fallback, et une flexibilité technologique accrue. Elle introduit cependant une complexité opérationnelle plus importante — notamment en termes de coordination entre services, de latence réseau et de gestion des transactions distribuées — ce qui constitue la transition naturelle vers une architecture microservices complète.